

Homework 2

Computer System Organization

ECE 361
Fall 2026

Overview

Homework 2 has you take a working one-file C program and split it into modules, without changing a single character of what it prints. Then you set up your term project. It is the topic of Tuesday's lecture: functions, scope, the call stack, recursion, and separate compilation. Homework 2 is due on Sun, Oct 11, 11:59pm, pushed to hw02/ and project/ in your course repository. AI tools are allowed. After the deadline, you sit a Shine about your code, due by Wed, Oct 14, 11:59pm, without any supporting materials.

Homework 2 is graded Satisfactory (S) / Not Yet (NY). It is Satisfactory when your repository meets every requirement listed at the end of this document at the deadline, and your Shine meets the bar. A Not Yet can be revised with a revision token within 3 days of receiving the grade.

Before you start: read Karumanchi 2.1–2.8 (recursion), and come to the studio on Thursday, Oct 8, where you draw the call stack of a recursive call and break a printed program into modules on paper. That is this homework, done by hand first.

Part 1: The starter program

Download hw02_starter.zip from Canvas and unpack it into hw02/. It holds:

Table 1. The files in the HW2 starter.

File	What it is
readings.c	one C file that reads sensor readings from standard input and prints a summary
data/	normal.txt, edge.txt, and empty.txt: three input files
expected/	what readings.c prints for each input file, and for edge.txt with a threshold of 25
regress.sh	the regression test: runs ./readings on every input file and compares its output with expected/

Build it and run it before you change anything:

```
gcc -std=c11 -Wall -Wextra -o readings readings.c
./readings < data/normal.txt
./readings 25 < data/edge.txt
./readings < data/normal.txt | diff - expected/normal.txt    # no output: identical
```

The program works. It is also one file with global variables, a main that does everything, and the same minimum, maximum, and mean loop written twice. Its recursive max_temp only works on the global array of temperatures. Read it and list what you would change before you change anything.

Part 2: Split it into modules

Rewrite the program as four files plus main.c, in hw02/:

Table 2. The modules of the refactored program.

Module	Files	Responsible for
reading	reading.h, reading.c	reading and parsing the input lines, skipping comments, blank lines, and bad lines
stats	stats.h, stats.c	minimum, maximum, and mean of an array, and the longest run above a threshold
histogram	histogram.h, histogram.c	printing the histogram
main	main.c	the command line, calling the modules, and printing the results

You design the interfaces: the function names, their parameters, and what each returns. Your design has to meet these rules:

1. **No global variables.** The readings live in `main` and are passed to the functions that need them.
2. **One recursive maximum.** Replace `max_temp` with a recursive function in `stats.c` that takes an array and its length, for example `float stats_max(const float a[], int n)`, and use it for the temperature and the humidity. The repeated loop disappears: each statistic is computed by one function, called twice.
3. **Headers are contracts.** Every header has an include guard, declares only what other files call, and has a comment for each function: what it takes, what it returns, and what it assumes (for example, $n \geq 1$).
4. **Helpers stay private.** A function used inside one `.c` file only is static. Name your public functions with the module as a prefix (`stats_max`, `histogram_print`) so that a reader knows where to look.
5. **Same output, byte for byte.** For every input file, your program prints exactly what expected/ holds. Refactoring changes the structure, never the behavior.

Draw your module diagram first: a box per module, an arrow from each module to the modules it calls. If two modules call each other, rethink the split. Your drawing from Thursday's studio is a good start.

Part 3: Build and test

- Write a Makefile. `make` builds readings from one `.o` file per `.c` file, using `gcc -std=c11 -Wall -Wextra` with no warnings. Each `.o` depends on its `.c` and on every header it includes, so that changing `stats.h` rebuilds every file that includes it, and changing `histogram.c` rebuilds only `histogram.o` and the program. `make clean` removes everything the build produced.
- Write `tests/test_stats.c`, unit tests for the `stats` module, with at least these four: one value; the maximum at the start and at the end; a value exactly equal to the threshold (it ends a run); and the recursive maximum on an array of 10,000 values. The test program prints PASS or FAIL for each test and a summary, and exits with a nonzero code if any test fails.
- `make test` builds and runs the unit tests, then runs the **regression test** from the starter: `sh regress.sh`. It compares your output with expected/ using `diff`; any difference is a failure.

Part 4: Your project, first steps

Your project sheet is on Shine, under My project. It is your specification for the whole term. This week you set up `project/`, so that every later homework has a place to land.

1. Save your project sheet as `project/ASSIGNMENT.md`.
2. Download the `iom361` folder from Canvas. Run `make` and `make test` in it once: if both work, your compiler and `iom361` work together. Copy `iom361_r4.h` and `iom361_r4.c` into `project/`.
3. Create the five modules of your device as stubs: `sensor.h`, `fsm.h`, `store.h`, `aux.h`, `report.h`, and their `.c` files. Each header has an include guard and a comment that says what the module will be responsible for in your device. Each `.c` includes its own header. No functions yet.
4. Write `main.c`: it calls `iom361_initialize` with the number of switches and LEDs on your sheet, prints the name of your device, and returns 1 if the initialization fails, 0 otherwise.
5. Write `project/Makefile`: `make` builds the program named on your sheet from `main.c`, the five modules, and `iom361_r4.c`, one `.o` per `.c`, with no warnings. `make clean` removes everything the build produced.
6. Start `project/SPEC.md`: your states, and a list of the events your device reacts to. The transition table comes in week 5.

README and AI usage

- hw02/README.md: how to build and test, and your module diagram (ASCII, or a photo of your drawing).
- hw02/AI_USAGE.md: which tools you used, for what, and one thing a model got wrong or that you had to fix, and how you found it. If you used no AI, say so.

Optional: bonus problem, one revision token

In Tuesday's lecture, the recursive maximum crashed at 200,000 values: every call that has not returned keeps a frame on the stack. This problem asks whether a different recursion can survive, and why.

1. Add `float stats_max_tail(const float a[], int n)` to your stats module: a recursive maximum whose helper carries the best value so far as a parameter, so that the recursive call is the last thing it does (a **tail call**). Test that it agrees with `stats_max`.
2. Write `tests/depth.c`, which runs either maximum on `n` values given on the command line. Build it with `-O0` and with `-O2`.
3. For both versions and both settings, find the largest `n` that does not crash, to within about 10 % (more than 20 million counts as "survives").
4. In `hw02/BONUS.md`: your four numbers in a table, with your computer and compiler, and an explanation in your own words. Why does one version survive at `-O2`? Why does it not at `-O0`? Why does `-O2` not save `stats_max`? What does this mean for a recursive function on a binary search tree?

The bonus earns one revision token when HW2 is Satisfactory and you answer the extra Shine question about your bonus well. AI tools are allowed, as for the rest of the homework, but the numbers must come from your own machine, and you must be able to explain them.

Deliverable

Push by Sun, Oct 11, 11:59pm. AI tools are allowed.

- hw02/Makefile, `main.c`, `reading.h`, `reading.c`, `stats.h`, `stats.c`, `histogram.h`, `histogram.c`
- hw02/tests/test_stats.c, and the starter's `regress.sh`, `data/`, and `expected/`
- hw02/README.md and hw02/AI_USAGE.md
- project/ASSIGNMENT.md, project/SPEC.md, project/Makefile, project/main.c, the five module stubs, `iom361_r4.h`, and `iom361_r4.c`
- Optional, for the bonus: `stats_max_tail` in `stats.c`, its tests, `tests/depth.c`, and `hw02/BONUS.md`

Shine

The Shine window opens on Monday, Oct 12, in the morning, and your Shine is due by Wednesday, Oct 14, 11:59pm. It takes about 10 to 15 minutes. Shine builds and tests your hw02/ and asks you about your own code. Expect the five kinds of question listed in the syllabus: explain, modify, expand, and debug.

Examples of the kind of question you may get:

- Your `stats_max` calls itself. Draw the call stack for an array of three values. What happens with an array of a million?
- Why is this helper static? What happens if you remove static and another file has a function with the same name?
- You change `stats.h`. Which files does make rebuild, and why?
- Which module of your project will look most like one of this week's modules, and what will it need that this one does not have?

If you did the bonus problem, your Shine has **one extra question about it**, about your own measurements.

HW2 Shine, due Wed, Oct 14, 11:59pm.

- No supporting materials: no notes, no book, no second screen, no AI, no other person.
- Webcam on. Shine records the session and notes when you leave its window.
- Do it on Monday or Tuesday, not late on Wednesday night.

Requirements for Satisfactory

- Every file in the Deliverable list above is in your repository at the deadline.
- In `hw02/`: make builds with no errors and no warnings, make test runs, all unit tests pass, the regression test finds no difference, and make clean works.
- No global variables, one recursive maximum, static helpers, and include guards.
- In `project/`: make builds your program with no warnings, and running it prints the name of your device.
- `README.md` and `AI_USAGE.md` are present and specific to your work.
- Your Shine is completed by the deadline and meets the bar.

Looking ahead

Next week is pointers. `reading_read` and `stats_longest_run_above` already return more than one result, and the way you solved that this week is exactly what HW3 is about. In your project, HW3 becomes the interfaces in `sensor.h` and `store.h`.